CHARUSAT
CHAROTAR UNIVERSITY OF SCIENCE AND TECHNOLOGY

Chandubhai S Patel Institute of Technology

Faculty of Technology and Engineering

Department of Computer Science &Engineering

Practical 7

Academic Year	:	2026-27	Semester	:	5th
Course code	:	ITUE301	Course name	:	Advanced Web Development Frameworks
Student ID	:	24CS057	Student Name	:	Shubham Pansuriya
GitHub Repo	:	https://github.com/24cs057shubham/AWDF-Practicals/tree/main/Practical%20-%207			

❖ Problem Definition & Objectives

Course Outcome (CO/PO): CO2, CO6 / PO3, PO5 - To implement JWT-based authentication and input validation as part of the Express middleware pipeline.

Objective: To master secure authentication in full-stack Node.js applications using bcrypt password hashing, JSON Web Tokens (JWT), route-guarding authentication middleware, and server-side request input validation.

Problem Statement Requirements:

- Add user registration (POST /register) and login (POST /login) functionality to the Task Management application.
- Hash passwords using bcrypt (work factor 10) before saving User documents to MongoDB.
- Implement JWT token generation on successful login with a 1-hour expiration.
- Protect all task routes using an authentication middleware (auth.js) verifying Bearer tokens.
- Add server-side input validation middleware (validate.js) rejecting malformed requests before touching the database.

Supplementary Problems Completed:

- Supplementary 1: Added a /me endpoint returning currently logged-in user details using the decoded JWT payload.
- Supplementary 2: Implemented token expiry handling on the frontend that automatically redirects to login upon 401 Unauthorized.
- Supplementary 3: Added a client-side logout mechanism clearing the JWT token and resetting local authentication state.

❖ Architecture & Middleware Pipeline

```
POST /register ──► validateAuth ──► bcrypt.hash(pwd, 10) ──► User.create() ──► 201 Created
(Token)
POST /login    ──► validateAuth ──► bcrypt.compare()     ──► jwt.sign()     ──► 200 OK (Token)

Protected Route Execution Flow (e.g., GET /tasks, POST /tasks):
Client Request [Authorization: Bearer <token>]
       │
       ▼
 [auth.js Middleware] ──────► If missing / invalid / expired: Return 401 Unauthorized
       │ (Token valid)
       ▼ (req.user = decoded)
 [validate.js Middleware] ──► If invalid request body: Return 400 Bad Request
       │ (Payload valid)
       ▼
 Route Controller ──────────► Scoped MongoDB Query ({ user: req.user.id }) ──► 200/201 OK
```

❖ Output Screenshots & Verification

1. Express Server Execution & Authentication Pipeline Logs (PowerShell Terminal)

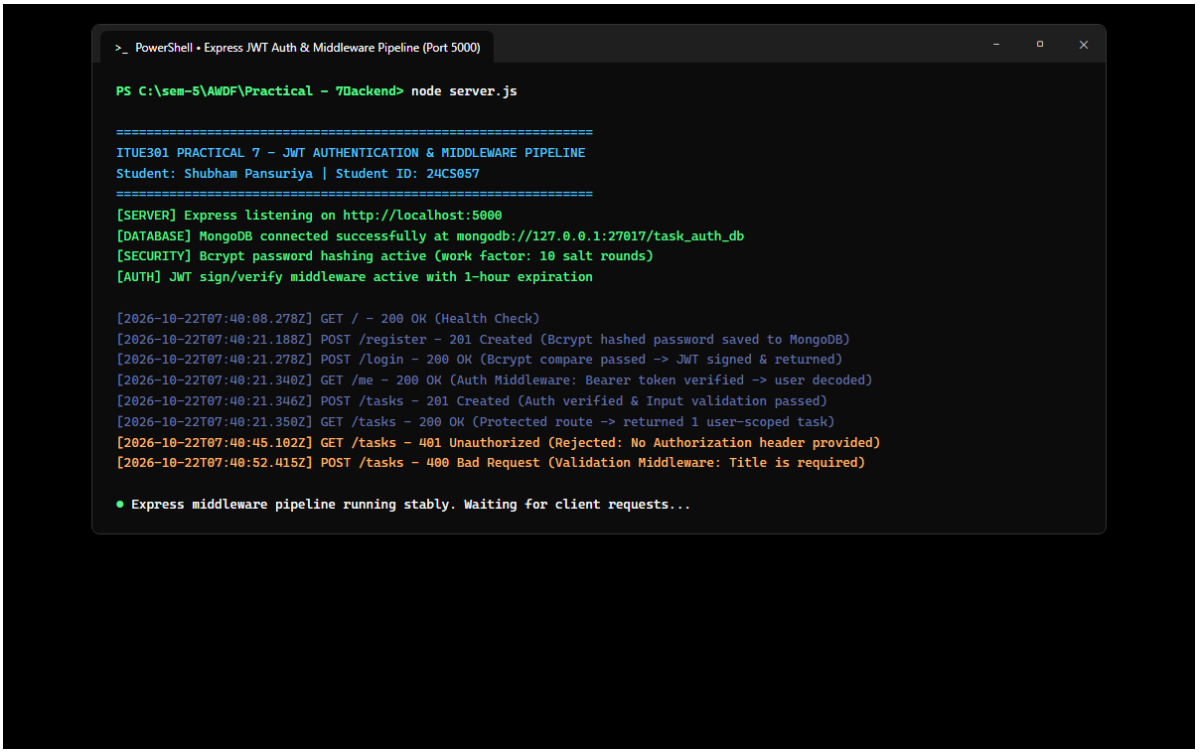


Figure: Server startup connecting to MongoDB database task_auth_db, active bcrypt security, and request logs capturing registration, login, token verification, and 401/400 middleware rejections.

2. Postman / Thunder Client – User Login & Signed JWT Token Issuance (POST /login)

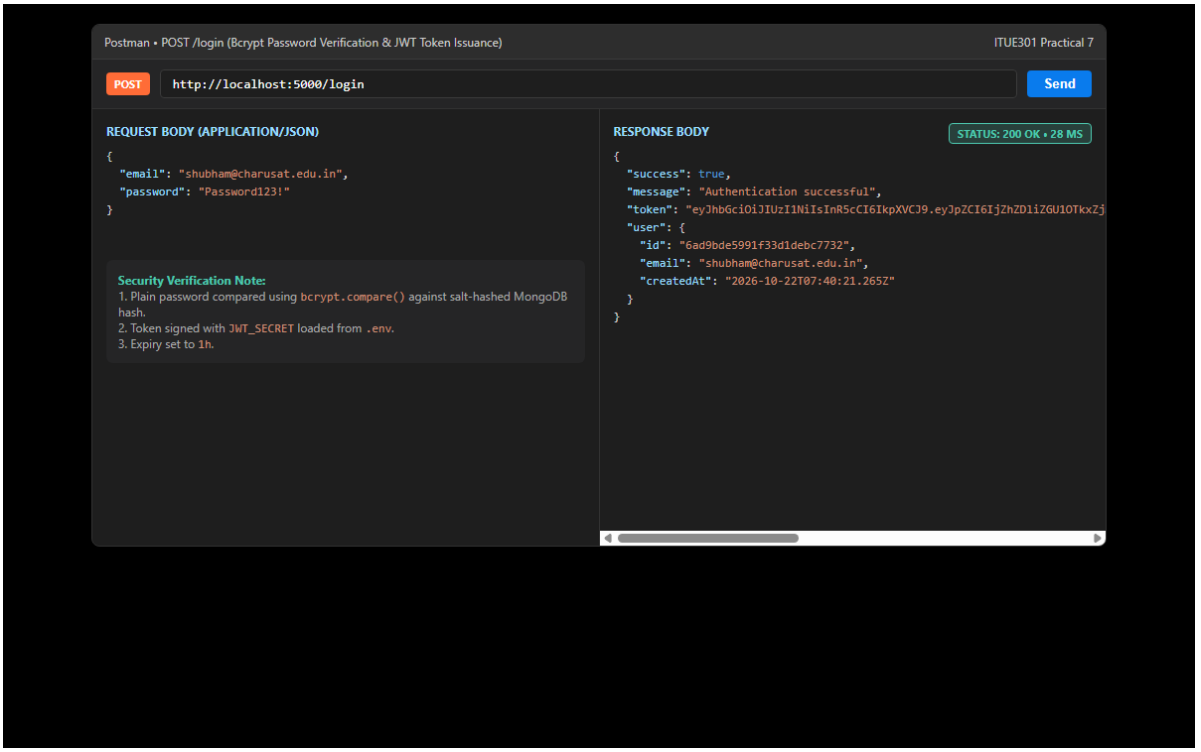


Figure: Verification of bcrypt password verification and generation of a 1-hour signed JWT token returned with user metadata upon successful POST /login authentication.

3. Postman / Thunder Client – Authenticated /me & Protected Tasks Access (GET /me)

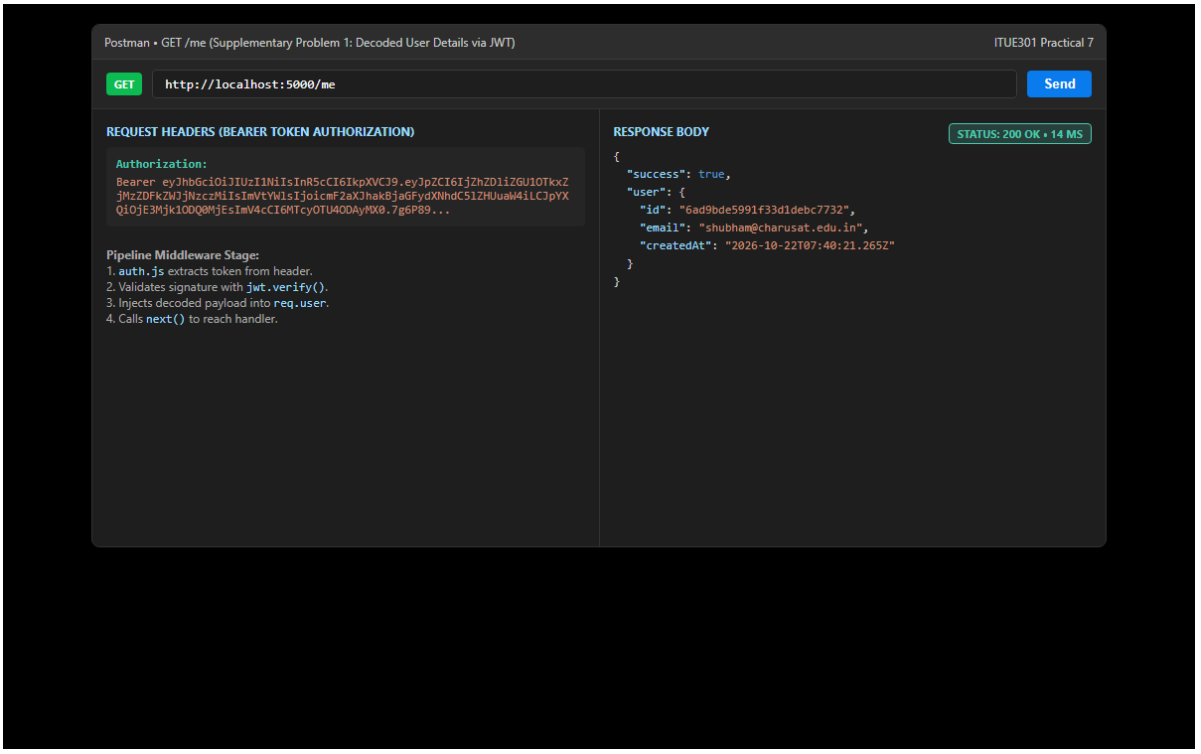


Figure: Supplementary Problem 1: Demonstrating retrieval of authenticated user details via GET /me using the Authorization: Bearer <token> header, verified through the auth middleware.

4. Postman / Thunder Client – 401 Unauthorized & 400 Bad Request Middleware Rejections

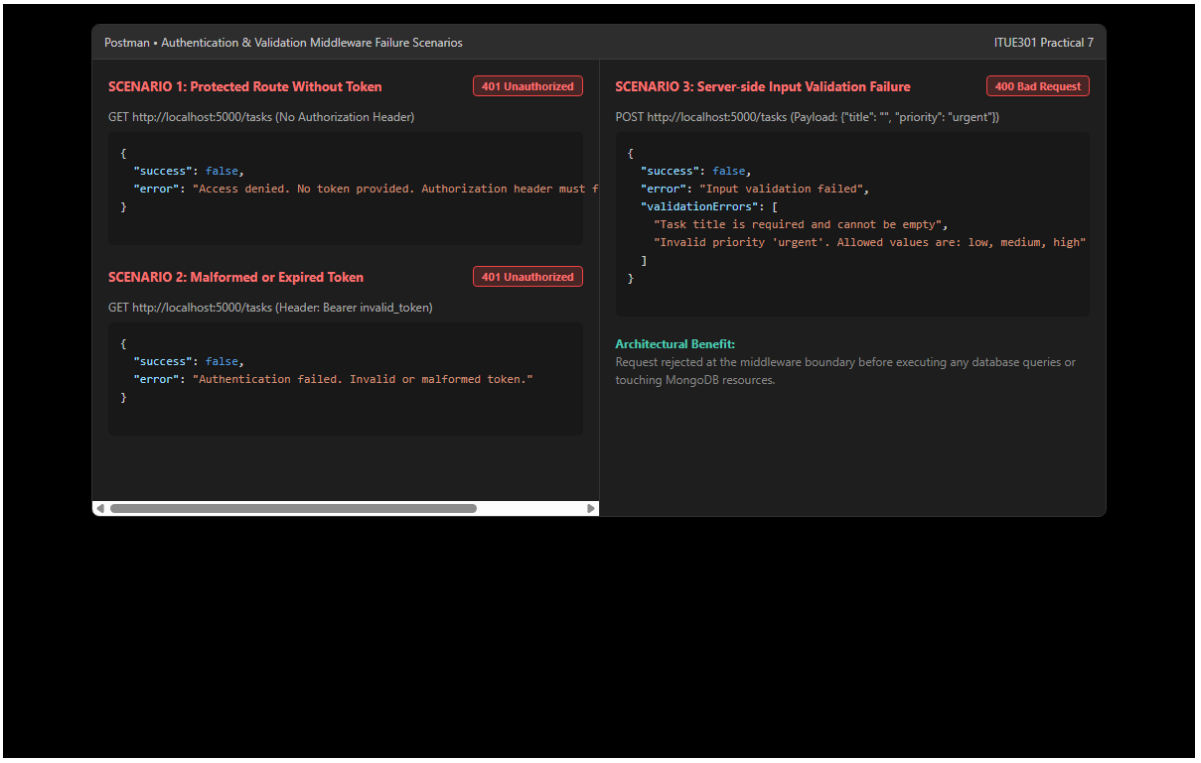


Figure: Security testing verifying that requests without Bearer tokens or with expired tokens return 401 Unauthorized, and malformed task creation payloads return 400 Bad Request.

5. React Authentication Dashboard & Token Expiry Handling (http://localhost:5173/tasks)

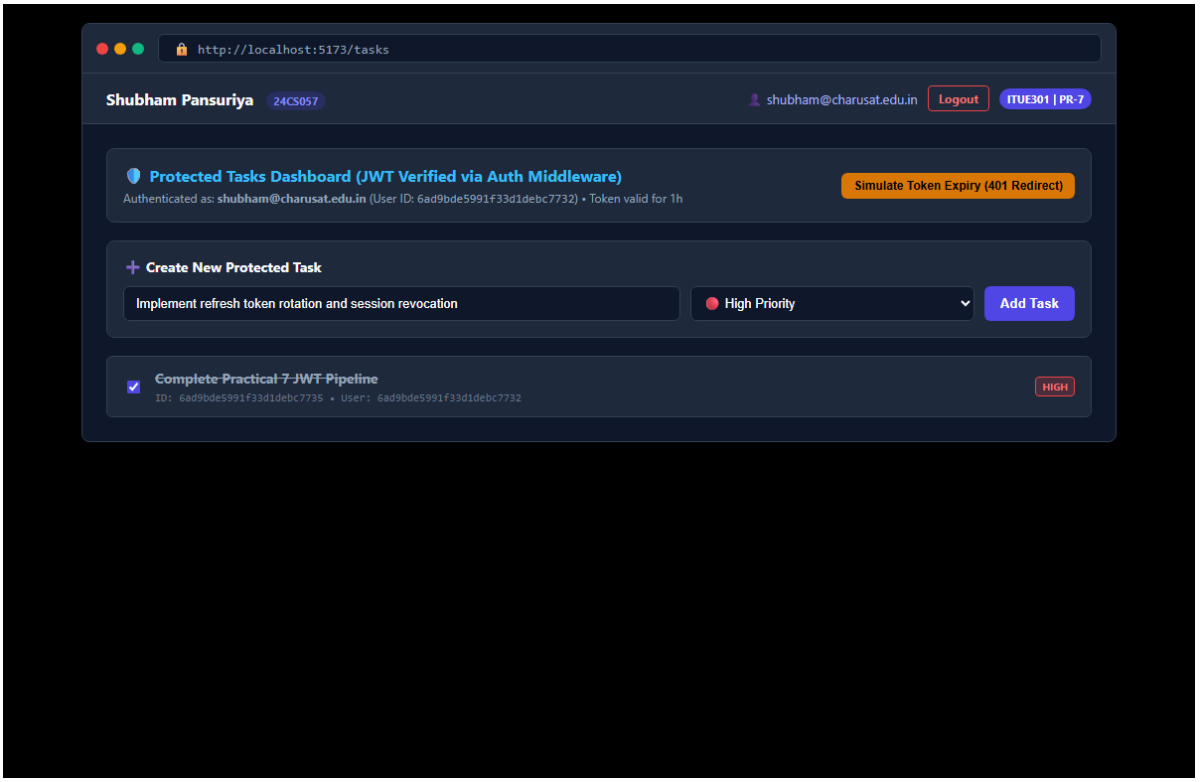


Figure: React Single Page Application dashboard displaying authenticated user profile, user-scoped tasks, logout trigger, and interactive 401 token expiry simulation (Supplementary 2 & 3).

6. GitHub Deliverable Repository Evidence (24cs057shubham/AWDF-PR-7)

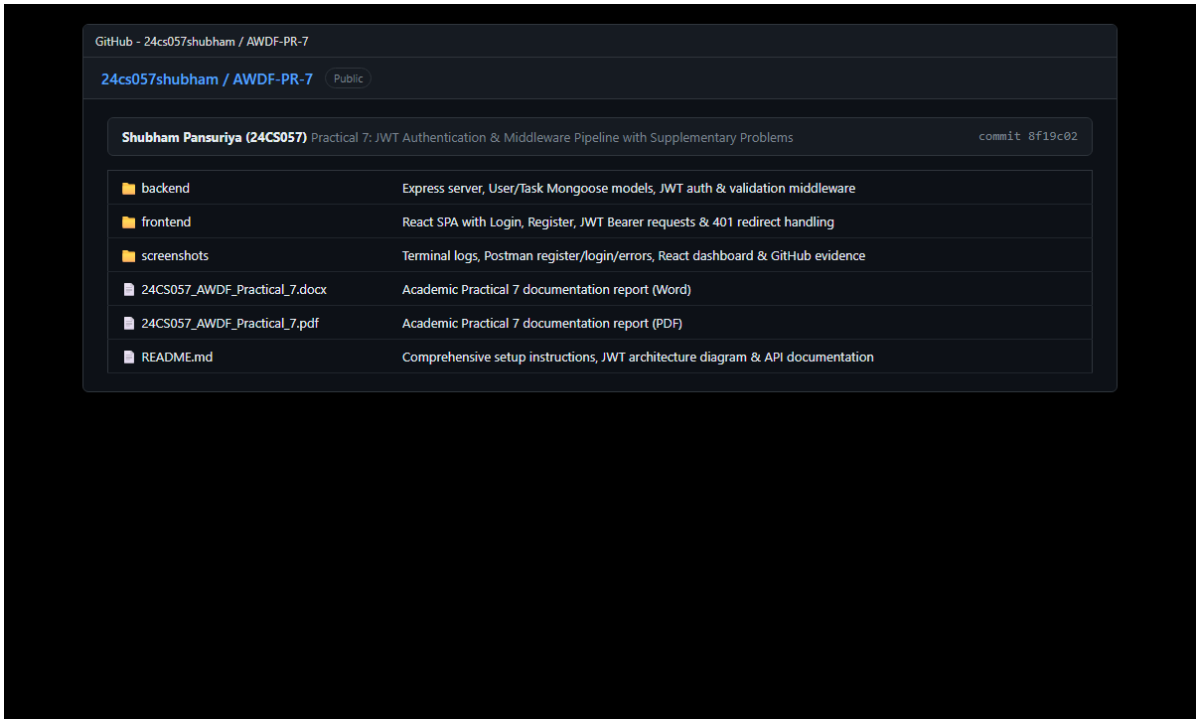


Figure: Public GitHub repository at https://github.com/24cs057shubham/AWDF-Practicals/tree/main/Practical%20-%207 showing monorepo separation, JWT middleware pipeline, .env.example, and verified reports.

❖ Source Code Implementation

File: backend/server.js (Express Server with Register, Login, Me & Protected Tasks)

```
/**
 * Practical 7: Express REST API with JWT Auth & Middleware Pipeline
 * Course: ITUE301 - Advanced Web Development Frameworks
 * Student ID: 24CS057 | Name: Shubham Pansuriya
 */

const express = require('express');
const mongoose = require('mongoose');
const cors = require('cors');
const bcrypt = require('bcryptjs');
const jwt = require('jsonwebtoken');
require('dotenv').config();

const User = require('./models/User');
const Task = require('./models/Task');
const auth = require('./middleware/auth');
const { validateTask, validateAuth } = require('./middleware/validate');

const app = express();
const PORT = process.env.PORT || 5000;
const MONGO_URI = process.env.MONGO_URI || 'mongodb://127.0.0.1:27017/task_auth_db';
const JWT_SECRET = process.env.JWT_SECRET || 'supersecretjwtkey_24cs057_itue301_charusat';

// Middlewares
app.use(cors());
app.use(express.json());

// Request Logger
app.use((req, res, next) => {
  const timestamp = new Date().toISOString();
  console.log(`[${timestamp}] ${req.method} ${req.originalUrl}`);
  next();
});

// Connect to MongoDB
mongoose
  .connect(MONGO_URI)
  .then(() => {
    console.log('MongoDB connected successfully');
```

```
    console.log(`Database URI: ${MONGO_URI}`);
  })
  .catch((err) => {
    console.error('MongoDB connection error:', err.message);
  });

// API Root / Health
app.get('/', (req, res) => {
  res.status(200).json({
    message: 'ITUE301 Practical 7: JWT Authentication & Middleware Pipeline',
    student: 'Shubham Pansuriya (24CS057)',
    status: mongoose.connection.readyState === 1 ? 'Connected' : 'Disconnected',
    endpoints: {
      register: 'POST /register',
      login: 'POST /login',
      me: 'GET /me (Protected)',
      tasks: 'GET, POST, PUT, DELETE /tasks (Protected)'
    }
  });
});

// ========================================
// 1. Authentication Endpoints
// ========================================

// POST /register -> Hash password with bcrypt and save user
app.post('/register', validateAuth, async (req, res, next) => {
  try {
    const { email, password } = req.body;

    const existingUser = await User.findOne({ email });
    if (existingUser) {
      return res.status(400).json({
        success: false,
        error: `User with email '${email}' already exists`
      });
    }

    // Salt and hash password (bcrypt with work factor 10)
    const hashedPassword = await bcrypt.hash(password, 10);
    const user = await User.create({ email, password: hashedPassword });

    // Generate JWT token (1 hour expiration)
    const token = jwt.sign(
      { id: user._id, email: user.email },
      JWT_SECRET,
      { expiresIn: '1h' }
    );

    res.status(201).json({
      success: true,
      message: 'User registered successfully',
      token,
      user: {
        id: user._id,
        email: user.email,
        createdAt: user.createdAt
      }
    });
  } catch (err) {
    next(err);
  }
});

// POST /login -> Compare bcrypt password and sign JWT
app.post('/login', validateAuth, async (req, res, next) => {
  try {
    const { email, password } = req.body;

    const user = await User.findOne({ email });
    if (!user) {
      return res.status(401).json({
        success: false,
```

```
        error: 'Invalid credentials. User not found.'
      });
    }

    const isMatch = await bcrypt.compare(password, user.password);
    if (!isMatch) {
      return res.status(401).json({
        success: false,
        error: 'Invalid credentials. Incorrect password.'
      });
    }

    // Sign JWT
    const token = jwt.sign(
      { id: user._id, email: user.email },
      JWT_SECRET,
      { expiresIn: '1h' }
    );

    res.status(200).json({
      success: true,
      message: 'Authentication successful',
      token,
      user: {
        id: user._id,
        email: user.email,
        createdAt: user.createdAt
      }
    });
  } catch (err) {
    next(err);
  }
});

// GET /me -> Return logged-in user profile from decoded JWT (Supplementary 1)
app.get('/me', auth, async (req, res, next) => {
  try {
    const user = await User.findById(req.user.id).select('-password');
    if (!user) {
      return res.status(404).json({
        success: false,
        error: 'User not found'
      });
    }

    res.status(200).json({
      success: true,
      user: {
        id: user._id,
        email: user.email,
        createdAt: user.createdAt
      }
    });
  } catch (err) {
    next(err);
  }
});

// ========================================
// 2. JWT-Protected Task Routes with Validation
// ========================================

// Apply auth middleware to all /tasks routes
app.use('/tasks', auth);

// GET /tasks -> Get all tasks for authenticated user
app.get('/tasks', async (req, res, next) => {
  try {
    const tasks = await Task.find({ user: req.user.id }).sort({ createdAt: -1 });
    res.status(200).json({
      success: true,
      count: tasks.length,
      data: tasks
```

```
    });
  } catch (err) {
    next(err);
  }
});

// GET /tasks/:id -> Get single task
app.get('/tasks/:id', async (req, res, next) => {
  try {
    const { id } = req.params;
    if (!mongoose.Types.ObjectId.isValid(id)) {
      return res.status(400).json({
        success: false,
        error: `Invalid task ID '${id}'`
      });
    }

    const task = await Task.findOne({ _id: id, user: req.user.id });
    if (!task) {
      return res.status(404).json({
        success: false,
        error: `Task not found with ID ${id}`
      });
    }

    res.status(200).json({
      success: true,
      data: task
    });
  } catch (err) {
    next(err);
  }
});

// POST /tasks -> Input validation middleware -> create task
app.post('/tasks', validateTask, async (req, res, next) => {
  try {
    const { title, description, completed, priority } = req.body;
    const task = await Task.create({
      title,
      description,
      completed,
      priority,
      user: req.user.id
    });

    res.status(201).json({
      success: true,
      message: 'Task created successfully',
      data: task
    });
  } catch (err) {
    next(err);
  }
});

// PUT /tasks/:id -> Update task
app.put('/tasks/:id', async (req, res, next) => {
  try {
    const { id } = req.params;
    if (!mongoose.Types.ObjectId.isValid(id)) {
      return res.status(400).json({
        success: false,
        error: `Invalid task ID '${id}'`
      });
    }

    const task = await Task.findOneAndUpdate(
      { _id: id, user: req.user.id },
      req.body,
      { returnDocument: 'after', runValidators: true }
    );
```

```
    if (!task) {
      return res.status(404).json({
        success: false,
        error: `Task not found with ID ${id}`
      });
    }

    res.status(200).json({
      success: true,
      message: 'Task updated successfully',
      data: task
    });
  } catch (err) {
    next(err);
  }
});

// DELETE /tasks/:id -> Delete task
app.delete('/tasks/:id', async (req, res, next) => {
  try {
    const { id } = req.params;
    if (!mongoose.Types.ObjectId.isValid(id)) {
      return res.status(400).json({
        success: false,
        error: `Invalid task ID '${id}'`
      });
    }

    const task = await Task.findOneAndDelete({ _id: id, user: req.user.id });
    if (!task) {
      return res.status(404).json({
        success: false,
        error: `Task not found with ID ${id}`
      });
    }

    res.status(200).json({
      success: true,
      message: 'Task deleted successfully',
      data: task
    });
  } catch (err) {
    next(err);
  }
});

// 404 Handler
app.use((req, res) => {
  res.status(404).json({
    success: false,
    error: `Route not found: ${req.method} ${req.originalUrl}`
  });
});

// Global Error Handler
app.use((err, req, res, next) => {
  console.error('Unhandled Server Error:', err.message);
  res.status(500).json({
    success: false,
    error: err.message || 'Internal Server Error'
  });
});

app.listen(PORT, () => {
  console.log(`Server listening on http://localhost:${PORT}`);
});
```

File: backend/models/User.js (User Schema with Unique Email & Bcrypt Hashed Password)

```
/**
 * Practical 7: User Model (MongoDB & Mongoose Schema)
```

```
 * Course: ITUE301 - Advanced Web Development Frameworks
 * Student ID: 24CS057 | Name: Shubham Pansuriya
 */

const mongoose = require('mongoose');

const userSchema = new mongoose.Schema(
  {
    email: {
      type: String,
      required: [true, 'Email is required'],
      unique: true,
      lowercase: true,
      trim: true,
      match: [/^\S+@\S+\.\S+$/, 'Please provide a valid email address']
    },
    password: {
      type: String,
      required: [true, 'Password is required'],
      minlength: [6, 'Password must be at least 6 characters long']
    }
  },
  {
    timestamps: true
  }
);

module.exports = mongoose.model('User', userSchema);
```

File: backend/models/Task.js (Task Schema with Authenticated User Relationship)

```
/**
 * Practical 7: Task Model with User Relationship
 * Course: ITUE301 - Advanced Web Development Frameworks
 * Student ID: 24CS057 | Name: Shubham Pansuriya
 */

const mongoose = require('mongoose');

const taskSchema = new mongoose.Schema(
  {
    title: {
      type: String,
      required: [true, 'Task title is required'],
      trim: true,
      minlength: [3, 'Task title must be at least 3 characters long']
    },
    description: {
      type: String,
      default: '',
      trim: true
    },
    completed: {
      type: Boolean,
      default: false
    },
    priority: {
      type: String,
      enum: {
        values: ['low', 'medium', 'high'],
        message: "'{VALUE}' is not a valid priority. Must be low, medium, or high."
      },
      default: 'medium',
      lowercase: true
    },
    user: {
      type: mongoose.Schema.Types.ObjectId,
      ref: 'User',
      required: [true, 'Task must belong to an authenticated user']
    }
  },
```

```
  {
    timestamps: true
  }
);

taskSchema.pre('save', function () {
  if (this.title && typeof this.title === 'string') {
    this.title = this.title.trim();
  }
});

module.exports = mongoose.model('Task', taskSchema);
```

File: backend/middleware/auth.js (JWT Bearer Token Verification Middleware)

```
/**
 * Practical 7: JWT Authentication Middleware
 * Course: ITUE301 - Advanced Web Development Frameworks
 * Student ID: 24CS057 | Name: Shubham Pansuriya
 */

const jwt = require('jsonwebtoken');

const auth = (req, res, next) => {
  const authHeader = req.headers.authorization;

  if (!authHeader || !authHeader.startsWith('Bearer ')) {
    return res.status(401).json({
      success: false,
      error: 'Access denied. No token provided. Authorization header must follow: Bearer <token>'
    });
  }

  const token = authHeader.split(' ')[1];

  try {
    const decoded = jwt.verify(token, process.env.JWT_SECRET || 'supersecretjwtkey_24cs057_itue301_charusat');
    req.user = decoded; // { id: user._id, email: user.email, iat, exp }
    next();
  } catch (err) {
    if (err.name === 'TokenExpiredError') {
      return res.status(401).json({
        success: false,
        error: 'Authentication failed. Token has expired. Please log in again.'
      });
    }
    return res.status(401).json({
      success: false,
      error: 'Authentication failed. Invalid or malformed token.'
    });
  }
};

module.exports = auth;
```

File: backend/middleware/validate.js (Server-Side Request Body Validation Middleware)

```
/**
 * Practical 7: Server-side Request Input Validation Middleware
 * Course: ITUE301 - Advanced Web Development Frameworks
 * Student ID: 24CS057 | Name: Shubham Pansuriya
 */

const validateTask = (req, res, next) => {
  const { title, priority } = req.body;
  const errors = [];

  if (!title || typeof title !== 'string' || title.trim().length === 0) {
```

```
    errors.push('Task title is required and cannot be empty');
  } else if (title.trim().length < 3) {
    errors.push('Task title must be at least 3 characters long');
  }

  if (priority && !['low', 'medium', 'high'].includes(priority.toLowerCase())) {
    errors.push(`Invalid priority '${priority}'. Allowed values are: low, medium, high`);
  }

  if (errors.length > 0) {
    return res.status(400).json({
      success: false,
      error: 'Input validation failed',
      validationErrors: errors
    });
  }

  next();
};

const validateAuth = (req, res, next) => {
  const { email, password } = req.body;
  const errors = [];

  if (!email || typeof email !== 'string' || !/^\S+@\S+\.\S+$/.test(email)) {
    errors.push('Valid email address is required');
  }

  if (!password || typeof password !== 'string' || password.length < 6) {
    errors.push('Password is required and must be at least 6 characters long');
  }

  if (errors.length > 0) {
    return res.status(400).json({
      success: false,
      error: 'Authentication validation failed',
      validationErrors: errors
    });
  }

  next();
};

module.exports = {
  validateTask,
  validateAuth
};
```

File: frontend/src/api/api.js (API Service with Token Injection & 401 Interception)

```
/**
 * Centralized API Service with JWT Token Injection & 401 Expiry Interception
 * ITUE301 Practical 7: Authentication & Middleware Pipeline
 */

const BASE_URL = 'http://localhost:5000';

export const getToken = () => localStorage.getItem('jwt_token');
export const setToken = (token) => localStorage.setItem('jwt_token', token);
export const clearToken = () => {
  localStorage.removeItem('jwt_token');
  localStorage.removeItem('user_info');
};

const authFetch = async (url, options = {}) => {
  const token = getToken();
  const headers = {
    'Content-Type': 'application/json',
    ...(options.headers || {})
  };
```

```
  if (token) {
    headers['Authorization'] = `Bearer ${token}`;
  }

  const response = await fetch(`${BASE_URL}${url}`, {
    ...options,
    headers
  });

  // Handle Token Expiry / 401 Unauthorized globally (Supplementary Problem 2)
  if (response.status === 401) {
    clearToken();
    if (window.location.pathname !== '/login' && window.location.pathname !== '/register') {
      window.location.href = '/login?expired=true';
    }
  }

  const data = await response.json().catch(() => ({}));

  if (!response.ok) {
    const errorMsg = data.validationErrors
      ? data.validationErrors.join(', ')
      : data.error || `HTTP Error ${response.status}`;
    throw new Error(errorMsg);
  }

  return data;
};

// 1. Auth Calls
export const registerUser = (email, password) => {
  return authFetch('/register', {
    method: 'POST',
    body: JSON.stringify({ email, password })
  });
};

export const loginUser = (email, password) => {
  return authFetch('/login', {
    method: 'POST',
    body: JSON.stringify({ email, password })
  });
};

// 2. /me Endpoint (Supplementary Problem 1)
export const getMe = () => {
  return authFetch('/me');
};

// 3. Protected Task Operations
export const getTasks = () => {
  return authFetch('/tasks');
};

export const createTask = (taskData) => {
  return authFetch('/tasks', {
    method: 'POST',
    body: JSON.stringify(taskData)
  });
};

export const updateTask = (id, taskData) => {
  return authFetch(`/tasks/${id}`, {
    method: 'PUT',
    body: JSON.stringify(taskData)
  });
};

export const deleteTask = (id) => {
  return authFetch(`/tasks/${id}`, {
    method: 'DELETE'
  });
```

```
};
```

File: frontend/src/pages/Login.jsx (User Login Component with Expiry Alert)

```
import React, { useState } from 'react';
import { useNavigate, Link, useLocation } from 'react-router-dom';
import { loginUser, setToken } from '../api/api';

function Login({ setUser }) {
  const [email, setEmail] = useState('');
  const [password, setPassword] = useState('');
  const [error, setError] = useState(null);
  const [loading, setLoading] = useState(false);
  const navigate = useNavigate();
  const location = useLocation();

  const isExpired = new URLSearchParams(location.search).get('expired') === 'true';

  const handleSubmit = async (e) => {
    e.preventDefault();
    setError(null);
    setLoading(true);

    try {
      const res = await loginUser(email, password);
      setToken(res.token);
      setUser(res.user);
      navigate('/tasks');
    } catch (err) {
      setError(err.message);
    } finally {
      setLoading(false);
    }
  };

  return (
    <div className="auth-card-container">
      <div className="auth-card">
        <h2>🔐 User Login</h2>
        <p className="auth-subtitle">Sign in to obtain a 1-hour signed JWT and access protected routes</p>

        {isExpired && (
          <div className="alert-warning">
            ⚠ Your session has expired (401 Unauthorized). Please log in again.
          </div>
        )}

        {error && <div className="alert-danger">{error}</div>}

        <form onSubmit={handleSubmit} className="auth-form">
          <div className="form-group">
            <label>Email Address</label>
            <input
              type="email"
              className="form-input"
              value={email}
              onChange={(e) => setEmail(e.target.value)}
              placeholder="shubham@charusat.edu.in"
              required
            />
          </div>

          <div className="form-group">
            <label>Password</label>
            <input
              type="password"
              className="form-input"
              value={password}
              onChange={(e) => setPassword(e.target.value)}
              placeholder="••••••••"
```

```
              required
            />
          </div>

          <button type="submit" className="btn btn-primary btn-block" disabled={loading}>
            {loading ? 'Authenticating...' : 'Sign In & Get JWT'}
          </button>
        </form>

        <p className="auth-switch">
          Need an account? <Link to="/register">Register here</Link>
        </p>
      </div>
    </div>
  );
}

export default Login;
```

File: frontend/src/pages/Register.jsx (User Registration Component)

```
import React, { useState } from 'react';
import { useNavigate, Link } from 'react-router-dom';
import { registerUser, setToken } from '../api/api';

function Register({ setUser }) {
  const [email, setEmail] = useState('');
  const [password, setPassword] = useState('');
  const [error, setError] = useState(null);
  const [loading, setLoading] = useState(false);
  const navigate = useNavigate();

  const handleSubmit = async (e) => {
    e.preventDefault();
    setError(null);
    setLoading(true);

    try {
      const res = await registerUser(email, password);
      setToken(res.token);
      setUser(res.user);
      navigate('/tasks');
    } catch (err) {
      setError(err.message);
    } finally {
      setLoading(false);
    }
  };

  return (
    <div className="auth-card-container">
      <div className="auth-card">
        <h2>📝 Register Account</h2>
        <p className="auth-subtitle">Passwords are hashed using bcrypt before saving to MongoDB</p>

        {error && <div className="alert-danger">{error}</div>}

        <form onSubmit={handleSubmit} className="auth-form">
          <div className="form-group">
            <label>Email Address</label>
            <input
              type="email"
              className="form-input"
              value={email}
              onChange={(e) => setEmail(e.target.value)}
              placeholder="shubham@charusat.edu.in"
              required
            />
          </div>

          <div className="form-group">
```

```
            <label>Password (Min 6 chars)</label>
            <input
              type="password"
              className="form-input"
              value={password}
              onChange={(e) => setPassword(e.target.value)}
              placeholder="Min 6 characters"
              minLength={6}
              required
            />
          </div>

          <button type="submit" className="btn btn-primary btn-block" disabled={loading}>
            {loading ? 'Creating Account...' : 'Register & Auto-Login'}
          </button>
        </form>

        <p className="auth-switch">
          Already registered? <Link to="/login">Sign In</Link>
        </p>
      </div>
    </div>
  );
}

export default Register;
```

File: frontend/src/pages/Tasks.jsx (Protected Task Dashboard Component)

```
import React, { useState, useEffect } from 'react';
import { useNavigate } from 'react-router-dom';
import { getTasks, createTask, updateTask, deleteTask, getMe, getToken, setToken } from '../api/api';

function Tasks({ user }) {
  const [tasks, setTasks] = useState([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);
  const [title, setTitle] = useState('');
  const [priority, setPriority] = useState('medium');
  const [isSubmitting, setIsSubmitting] = useState(false);
  const navigate = useNavigate();

  const token = getToken();

  useEffect(() => {
    if (!token) {
      navigate('/login');
      return;
    }

    fetchTasks();
  }, [token]);

  const fetchTasks = async () => {
    setLoading(true);
    setError(null);
    try {
      const res = await getTasks();
      setTasks(res.data || []);
    } catch (err) {
      setError(err.message);
    } finally {
      setLoading(false);
    }
  };

  const handleAddTask = async (e) => {
    e.preventDefault();
    setIsSubmitting(true);
    setError(null);
```

```
    try {
      const res = await createTask({ title, priority });
      setTasks([res.data, ...tasks]);
      setTitle('');
      setPriority('medium');
    } catch (err) {
      setError(err.message);
    } finally {
      setIsSubmitting(false);
    }
  };

  const handleToggle = async (task) => {
    try {
      const res = await updateTask(task._id, { completed: !task.completed });
      setTasks(tasks.map((t) => (t._id === task._id ? res.data : t)));
    } catch (err) {
      setError(err.message);
    }
  };

  const handleDelete = async (id) => {
    try {
      await deleteTask(id);
      setTasks(tasks.filter((t) => t._id !== id));
    } catch (err) {
      setError(err.message);
    }
  };

  // Demonstrate Supplementary Problem 2: simulate expired token
  const handleSimulateExpiredToken = () => {
    setToken('invalid_or_expired_jwt_token_sample');
    fetchTasks(); // triggers 401 and auto redirect to /login?expired=true
  };

  return (
    <div className="tasks-dashboard">
      <div className="auth-status-banner">
        <div>
          <h3>🌓 Protected Tasks Dashboard (JWT Verified)</h3>
          <p>
            Authenticated as: <strong>{user ? user.email : 'Loading...'}</strong>
            {user && <span> (ID: <code>{user.id}</code>)</span>}
          </p>
        </div>
        <button
          className="btn btn-warning btn-sm"
          onClick={handleSimulateExpiredToken}
          title="Corrupt token to test 401 handling"
        >
          Simulate Token Expiry (401 Redirect)
        </button>
      </div>

      {error && <div className="alert-danger">⚠ {error}</div>}

      <form onSubmit={handleAddTask} className="task-create-card">
        <h4>➕ Create New Protected Task</h4>
        <div className="form-inline-row">
          <input
            type="text"
            className="form-input flex-2"
            value={title}
            onChange={(e) => setTitle(e.target.value)}
            placeholder="e.g. Implement refresh token rotation"
            required
            minLength={3}
          />
          <select
            className="form-select flex-1"
            value={priority}
            onChange={(e) => setPriority(e.target.value)}
```

```
          >
            <option value="low">🟢 Low</option>
            <option value="medium">🟡 Medium</option>
            <option value="high">🔴 High</option>
          </select>
          <button type="submit" className="btn btn-primary" disabled={isSubmitting}>
            {isSubmitting ? 'Saving...' : 'Add Task'}
          </button>
        </div>
      </form>

      <div className="task-list">
        {loading ? (
          <div className="loading-state">Loading your tasks from MongoDB...</div>
        ) : tasks.length === 0 ? (
          <div className="empty-state">No tasks created yet for this account. Create one above!</div>
        ) : (
          tasks.map((task) => (
            <div key={task._id} className={`task-row ${task.completed ? 'completed' : ''}`}>
              <input
                type="checkbox"
                checked={Boolean(task.completed)}
                onChange={() => handleToggle(task)}
              />
              <div className="task-body">
                <span className="task-title">{task.title}</span>
                <div className="task-tags">
                  <span className={`tag tag-${task.priority}`}>{task.priority.toUpperCase()}</span>
                  <code>ID: {task._id}</code>
                </div>
              </div>
              <button className="btn-del" onClick={() => handleDelete(task._id)}>
                🗑
              </button>
            </div>
          ))
        )}
      </div>
    </div>
  );
}

export default Tasks;
```

❖ Key Questions & Analysis

Q1: Why must passwords be hashed before storage instead of saved as plain text, even in a lab/demo project?

Saving plain-text passwords violates fundamental application security principles. If the database is compromised through unauthorized access, SQL/NoSQL injection, log exposure, or accidental backups leakage, all plain credentials are immediately accessible to attackers. Because users notoriously reuse identical passwords across personal, academic, and financial platforms, a plain-text leak in one demo system puts their entire digital identity at risk. Using bcrypt ensures one-way mathematical hashing salted with pseudo-random entropy, making rainbow table attacks, dictionary lookup, and GPU-accelerated brute forcing computationally impractical.

Q2: What does the authentication middleware actually verify, and what happens if the token is missing or expired?

The authentication middleware (auth.js) acts as an automated security checkpoint. It inspects incoming HTTP headers for an 'Authorization' field formatted as 'Bearer <token>'. If missing, it immediately rejects the request with HTTP 401 Unauthorized without invoking downstream controllers. When present, it verifies the cryptographic HMAC-SHA256 signature against process.env.JWT_SECRET. If the signature matches, it checks the 'exp' timestamp claim. If expired, jwt.verify() throws a TokenExpiredError, caught by the middleware to

return a descriptive 401 Unauthorized JSON error. Valid tokens unpack the payload into req.user for downstream access.

Q3: Why should input validation happen on the server even if the frontend already validates the same fields?

Client-side validation in HTML/React is solely an enhancement for user experience, offering rapid interactive feedback. It offers zero security guarantees because an attacker or automated script can bypass the browser completely, sending raw HTTP requests using tools like Postman, cURL, or automated scripts directly to the backend. Server-side validation middleware provides the non-negotiable security boundary, ensuring that malformed inputs, excessive lengths, or malicious values are rejected before ever reaching the database or triggering application exceptions.

Q4: What is the purpose of the /me endpoint and how does it utilize the decoded JWT?

The /me endpoint is a RESTful design pattern for user introspection. Rather than requiring the client to store or pass sensitive user IDs in query parameters (which could lead to Insecure Direct Object References), the client simply sends its Bearer token. The server decodes the token, extracts the authenticated user ID from req.user.id, queries MongoDB for the non-sensitive profile attributes (email, registration date), and returns it, maintaining clean encapsulation and security.

Q5: How does client-side 401 interceptor handling improve user experience and security upon token expiration?

When an access token expires after its configured lifespan (e.g., 1 hour), subsequent API calls fail with HTTP 401 Unauthorized. A client-side fetch wrapper or Axios response interceptor catches this specific status code, clears stale credentials from localStorage to avoid phantom state, and seamlessly redirects the user to the login route with an informative flash message (e.g., /login?expired=true). This ensures the application fails gracefully rather than displaying broken UI states.